SupermarketApp

De la primera clonación a la presentación del proyecto

Guía paso a paso para Windows

Git y GitHub · Java 21 · Swing · Oracle XE 21c

Instala las herramientas, prepara tu propia base, compila, ejecuta y demuestra el ciclo de caja. Incluye comandos, resultados esperados y soluciones a errores frecuentes.

Para estudiantes que comienzan con Git y Java

github.com/MJMontufar/SupermarketApp

Edición 1.0 · 27 de octubre de 2026

Índice y forma de uso

Sigue los apartados 01 a 16 para instalar y presentar en un equipo nuevo. Para actualizar una copia funcional, ve al 17. Los apartados 18 y 19 son opcionales para colaboración y pruebas.

01. Antes de empezar: el recorrido completo 3
02. Las herramientas y las contraseñas 4
03. Aprende a reconocer la terminal 5
04. Instalar Git y comprobar el acceso a GitHub 6
05. Clonar el repositorio por primera vez 7
06. Instalar y seleccionar Java 21 8
07. Abrir correctamente el proyecto en VS Code 9
08. Instalar Oracle XE 21c en Windows 10
09. Conectarse a la base correcta 11
10. Crear POS y POS_TEST una sola vez 12
11. Crear las tablas y cargar los datos iniciales 13
12. Descargar el driver y probar la conexión 14
13. Compilar, abrir e ingresar como administrador 15
14. Crear usuarios, conocer roles y añadir fotos 16
15. Demostrar administración, roles y catálogos 17
16. Presentación: ventas, retiro, reversión y cierre 18
17. Actualizar el código sin repetir la instalación 19
18. Compartir una mejora mediante una rama y un PR 20
19. Pruebas Java y verificaciones opcionales 21
20. Capturas y archivos para presentar 22
21. Respaldo del proyecto, fotos y capturas 23
22. Exportar los datos de Oracle con Data Pump 24
23. Explicar el código y abrir el diagrama ER 25
24. Solucionar errores de Git, rutas y Java 26
25. Solucionar errores Oracle y comprobar la entrega 27

Los recuadros identifican si un comando va en PowerShell (PS>) o SQL*Plus (SQL>). No copies esos indicadores. El PDF permite seleccionar texto; revisa que no se añadan saltos de línea al pegar comandos.

01. Antes de empezar: el recorrido completo

Esta guía permite instalar una copia independiente del supermercado en otra computadora Windows, descargar el código con Git, compilarlo y demostrarlo. Está dirigida a personas que comienzan con Git, Java y Oracle.

Ruta para una computadora nueva: instala Git, Java 21, VS Code y Oracle; obtén acceso a GitHub; clona el repositorio; prepara POS; descarga el driver; comprueba la conexión y abre la aplicación. Sigue los apartados en orden.

Ruta para una computadora ya preparada: comprueba las versiones, entra al clon existente y actualiza según el apartado 17. No repitas la creación de usuarios ni tablas. El esquema de pruebas se prepara solamente si lo usarás.

Concepto	Significado en este proyecto
Git	Programa que registra versiones del código en tu computadora.
GitHub	Sitio donde se comparte el repositorio y se revisan cambios.
Repositorio	Carpeta del proyecto con su historial Git.
Clone	Primera descarga del repositorio y su historial.
Pull	Incorporación de cambios del repositorio remoto a tu copia.
Commit	Versión registrada localmente con un mensaje.
Push	Envío de commits locales a GitHub.
Rama	Línea de trabajo; `main` contiene la versión integrada.
Pull request o PR	Solicitud para revisar e incorporar una rama.

Git no descarga Oracle ni Java. Tampoco sincroniza la base de datos. Cada equipo tendrá sus propias cuentas técnicas, usuarios de la aplicación, ventas, existencias y fotografías, salvo que se restaure un respaldo compatible.

02. Las herramientas y las contraseñas

Descarga las herramientas desde sus sitios oficiales:

Herramienta	Dirección y selección
Git para Windows	https://git-scm.com/downloads/win
Temurin JDK 21 LTS	https://adoptium.net/temurin/releases/?version=21 — Windows, x64, JDK, MSI.
Visual Studio Code	https://code.visualstudio.com/
Oracle Database XE 21c	https://www.oracle.com/database/technologies/xe-downloads.html — Windows x64.
Repositorio	https://github.com/MJMontufar/SupermarketApp

Utiliza Windows de 64 bits y una cuenta con permisos para instalar programas. Reserva un margen práctico de 15–20 GB libres para Oracle, sus temporales y datos; no es una declaración del mínimo oficial. Conserva espacio adicional para respaldos. Una descarga de código por sí sola no deja el sistema listo.

Credencial	Para qué sirve	Quién la define
Cuenta GitHub	Acceso al repositorio y colaboración.	Cada compañero.
SYSTEM de Oracle	Crear y administrar esquemas locales.	Instalador de Oracle.
POS de Oracle	Conexión de la aplicación con sus tablas.	Script SQL 00.
POS_TEST de Oracle	Conexión a tablas separadas de pruebas.	Script SQL 00.
admin de la aplicación	Entrar al menú del supermercado.	Datos iniciales: `Admin123!`.
ana, luis, marta	Usuarios del escenario de presentación.	Se crean desde la aplicación.

`admin` no es un usuario de Oracle. La contraseña pública de entrenamiento `Admin123!` no debe usarse como contraseña personal ni como clave técnica. Guarda tus claves locales de forma privada; no las escribas en GitHub.

03. Aprende a reconocer la terminal

Abre **Inicio → Windows PowerShell**. También puedes usar la terminal integrada de VS Code mediante **Terminal → New Terminal**, seleccionando PowerShell.

Lo que ves	Dónde estás	Qué acepta
`PS C:\Users\...>`	PowerShell	`git`, `java`, `sqlplus` y archivos `.ps1`.
`SQL>`	SQL*Plus conectado a Oracle	SQL, `SHOW USER`, `CONNECT`, `@archivo.sql`.
`>>`	PowerShell espera terminar una instrucción	Revisa comillas o cancela la instrucción incompleta con Ctrl+C.

Copia solamente los comandos de los recuadros. No escribas `PS ...>` ni `SQL>` delante de ellos. Ejecuta cada bloque en la terminal indicada.

Estos comandos de PowerShell ayudan a orientarte:

POWERSHELL · PS>

```
Get-Location
Get-ChildItem
Set-Location "$env:USERPROFILE"
```

El primero muestra la carpeta actual; el segundo lista su contenido; el tercero entra en tu carpeta de usuario. `$env:USERPROFILE` se adapta al nombre de tu cuenta: no copies la ruta del otro compañero. Usa comillas cuando haya espacios.

En SQL*Plus, sal con `EXIT` para volver a PowerShell. Durante una solicitud de contraseña, escribir no muestra letras ni asteriscos: es normal. Escribe la clave y pulsa Enter una vez. No publiques capturas con claves visibles.

04. Instalar Git y comprobar el acceso a GitHub

1. Instala Git para Windows. Conserva Git Credential Manager y la opción que permite usar Git desde la línea de comandos y otros programas.

2. Cierra y vuelve a abrir PowerShell después de instalar.

3. Comprueba la instalación:

POWERSHELL · PS>

```
git --version
git credential-manager --version
```

Debe aparecer una versión en cada caso. Si `git` no se reconoce, abre una terminal nueva o revisa la instalación antes de continuar.

4. Crea tu propia cuenta en https://github.com/ e inicia sesión en el navegador.

5. Abre https://github.com/MJMontufar/SupermarketApp con esa misma cuenta.

Si el repositorio es privado: el propietario entra en el repositorio, abre **Settings → Collaborators** o **Collaborators and teams**, selecciona **Add people** e invita tu usuario GitHub. Debes aceptar la invitación y poder ver los archivos antes de clonar. No compartan la contraseña del propietario.

6. Si más adelante vas a crear commits, configura tu identidad sustituyendo los dos valores de ejemplo por los tuyos. No son credenciales de acceso:

POWERSHELL · PS>

```
git config --global user.name "Tu nombre"
git config --global user.email "tu-correo@example.com"
```

Puedes usar el correo privado `noreply` que GitHub muestra en Settings → Emails. Si solo vas a descargar y ejecutar, esa configuración no es necesaria.

05. Clonar el repositorio por primera vez

En PowerShell, crea una carpeta corta fuera de Descargas y de carpetas sincronizadas. Así evitamos rutas excesivamente largas y confusiones de ZIP:

POWERSHELL · PS>

```
New-Item -ItemType Directory -Path "$env:USERPROFILE\Proyectos" -Force
Set-Location "$env:USERPROFILE\Proyectos"
git clone https://github.com/MJMontufar/SupermarketApp.git SupermarketApp-Git
```

Si Git abre el navegador, autoriza tu propia cuenta GitHub con acceso al repositorio. GitHub no admite la contraseña de la cuenta como autenticación HTTPS de operaciones Git. Utiliza el acceso del navegador con Credential Manager.

Si aparece `Invalid username or token` o `Authentication failed`, ejecuta:

POWERSHELL · PS>

```
git credential-manager configure
git credential-manager github login --browser --force
git ls-remote https://github.com/MJMontufar/SupermarketApp.git HEAD
```

El último comando debe devolver un identificador y `HEAD`. Después repite el clone únicamente si la carpeta de destino no existe. Si quedó una carpeta parcial, revísala y renómbrala para conservarla; no borres otra copia funcional.

Tras clonar correctamente:

POWERSHELL · PS>

```
Set-Location .\SupermarketApp-Git
git status
git remote -v
Get-ChildItem
```

Lo normal es estar en `main`, actualizado con `origin/main`, y ver `nothing to commit, working tree clean`. `origin` es el nombre del remoto. Si la carpeta ya era un clon válido, entra en ella y utiliza `git pull`, no clone.

06. Instalar y seleccionar Java 21

1. Descarga **Temurin JDK 21 LTS, Windows x64, MSI**. Debe ser JDK, no solamente JRE.

2. En el instalador activa **Añadir al PATH** y **Establecer JAVA_HOME**. La integración de registro JavaSoft no es necesaria para este proyecto.

3. Termina la instalación y abre una PowerShell nueva. Comprueba:

POWERSHELL · PS>

```
java -version
javac -version
$env:JAVA_HOME
where.exe java
where.exe javac
```

`java` y `javac` deben indicar **21**. `JAVA_HOME`, si está definido, apunta a la carpeta raíz del JDK: no termina en `bin` ni en `java.exe`.

Si sigue apareciendo Java 17, hay otra instalación antes en PATH. Abre **Editar las variables de entorno de esta cuenta**, revisa `JAVA_HOME` y coloca `%JAVA_HOME%\bin` antes de las entradas antiguas de Java. Si Windows da prioridad al PATH del sistema y la cuenta de usuario no puede cambiarlo, pide al administrador que ordene esa entrada; no borres carpetas de Java manualmente.

Para probar solo en la terminal actual, después de corregir `JAVA_HOME`:

POWERSHELL · PS>

```
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
java -version
javac -version
```

Este último ajuste no es permanente. Valida también en una terminal nueva. Si Windows no actualiza las variables al reabrir terminales, cierra sesión y vuelve a entrar. No cambies el código para compilar con Java 17.

07. Abrir correctamente el proyecto en VS Code

1. Instala **Visual Studio Code**. Es un programa distinto de Visual Studio.

2. En VS Code abre Extensiones con Ctrl+Mayús+X e instala **Extension Pack for Java**, de Microsoft. Incluye el soporte de lenguaje Java de Red Hat.

3. Selecciona **File → Open Folder** y abre la carpeta `SupermarketApp-Git`. No abras solamente `src`, `com`, `scripts` ni el directorio padre `Proyectos`.

4. Espera a que Java termine de iniciar y acepta la confianza del espacio de trabajo únicamente si reconoces este repositorio.

Debes ver estas rutas en el explorador:

RESULTADO O REFERENCIA

```
SupermarketApp-Git/
  .vscode/
  config.properties
  docs/
  lib/
  scripts/
  sql/
  src/com/supermercado/pos/Main.java
  test/
```

Abre **Terminal → New Terminal**. Para situarte siempre en la raíz:

POWERSHELL · PS>

```
Set-Location "$env:USERPROFILE\Proyectos\SupermarketApp-Git"
Test-Path .\scripts\iniciar.ps1
Test-Path .\src\com\supermercado\pos\Main.java
```

Ambos resultados deben ser `True`. `.vscode/settings.json` configura `src` y `test` como carpetas de código y `lib/**/*.jar` como bibliotecas.

Si el proyecto compila en consola pero el editor marca imports incorrectos, usa Ctrl+Mayús+P → **Java: Clean Java Language Server Workspace** y reinicia. Evita tener dos servidores de lenguaje Java compitiendo; si aparece el conflicto conocido con la extensión Oracle Java, desactívala solo en este workspace y conserva el soporte de Red Hat. No cambies los paquetes Java.

08. Instalar Oracle XE 21c en Windows

1. Si Oracle XE ya funciona en este equipo, no lo reinstales: pasa al apartado 09.

2. Comprueba el espacio libre en Configuración → Sistema → Almacenamiento. Libera temporales o aplicaciones que conozcas; conserva documentos y respaldos.

3. Descarga el ZIP de **Oracle Database XE 21c para Windows x64** del sitio oficial. Extrae completamente a una ruta corta, por ejemplo `C:\Instaladores\OracleXE`.

4. Ejecuta el instalador desde la carpeta extraída, con permisos de administrador.

5. Define una contraseña de instalación y guárdala. Para evitar el problema de análisis del instalador que vimos en este proyecto, usa una clave larga con mayúsculas, minúsculas y números. No incluyas la clave en capturas.

6. Espera a que termine la creación de la base. Un mensaje de fallo al configurar la base no equivale a una instalación completa: conserva el error y sus logs.

No confundas **Oracle Database** con SQL Developer: SQL Developer es un cliente y no sustituye al servidor. Para esta guía SQL*Plus y Data Pump bastan.

En PowerShell comprueba los servicios y el puerto:

POWERSHELL · PS>

```
Get-Service -Name '*Oracle*' | Select-Object Name, Status
Test-NetConnection localhost -Port 1521
Get-Command sqlplus
```

En una instalación estándar debe existir `OracleServiceXE` y un listener; `TcpTestSucceeded` debe ser `True`. El nombre exacto del listener puede variar. Si SQL*Plus no está en PATH, localiza `sqlplus.exe` en la carpeta `bin` de Oracle y añade esa ruta al PATH o ejecuta el programa por su ruta completa entre comillas precedida por `&`. No compartas el servidor de otro equipo para estos pasos.

09. Conectarse a la base correcta

Desde PowerShell, entra en el proyecto y abre SQL*Plus:

POWERSHELL · PS>

```
Set-Location "$env:USERPROFILE\Proyectos\SupermarketApp-Git"
sqlplus -L system@//localhost:1521/XEPDB1
```

Escribe la contraseña **SYSTEM** elegida en la instalación local. No escribas la contraseña de GitHub ni `Admin123!`. Si falla, vuelve a PowerShell y revisa el usuario de la conexión antes de cambiar contraseñas.

Ahora debes ver `SQL>`. Ejecuta:

SQL*PLUS · SQL>

```
SHOW USER
SHOW CON_NAME
SELECT username FROM dba_users WHERE username IN ('POS', 'POS_TEST');
```

Resultados esperados: usuario **SYSTEM** y contenedor **XEPDB1**.

Si la consulta no devuelve POS ni POS_TEST: sigue con el apartado 10. **Si devuelve cuentas existentes:** no ejecutes SQL 00. Confirma sus claves y consulta las tablas desde la cuenta correspondiente según el apartado 11. Si aparece solamente una cuenta, hay preparación parcial: conserva la base y revisa el estado antes de ejecutar otros scripts.

La sesión de SQL*Plus conserva la carpeta desde la que se abrió. Por eso abrimos SQL*Plus desde la raíz: los archivos `sql/...` se encuentran allí.

10. Crear POS y POS_TEST una sola vez

Este paso requiere estar en `SQL>` como **SYSTEM en XEPDB1** y haber confirmado que POS y POS_TEST no existen en este equipo. Ejecuta:

SQL*PLUS · SQL>

```
@"sql/00_crear_usuarios.sql"
```

El script crea dos cuentas. `PASSWORD pos` y `PASSWORD pos_test` piden definir y confirmar sus respectivas claves sin mostrarlas. Elige claves distintas y guárdalas de forma privada. No hay que escribirlas dentro de los SQL.

Una vez terminado, sal:

SQL*PLUS · SQL>

```
EXIT
```

Ahora estás otra vez en PowerShell. Conéctate a POS:

POWERSHELL · PS>

```
sqlplus -L pos@//localhost:1521/XEPDB1
```

Escribe la clave recién definida para **POS**. Comprueba en SQL*Plus:

SQL*PLUS · SQL>

```
SHOW USER
SHOW CON_NAME
SELECT COUNT(*) AS tablas FROM user_tables;
```

Debes ver **POS**, **XEPDB1** y **0 tablas** antes de crear un esquema nuevo. Cambiar una contraseña con `PASSWORD pos` desde SYSTEM no cambia la cuenta de la sesión; por eso salimos y abrimos una conexión POS explícita.

Si el script se interrumpió después de crear las cuentas pero antes de definir sus claves, SYSTEM puede usar `PASSWORD pos` y `PASSWORD pos_test` para completar las claves. No repitas la creación ni borres cuentas que ya tengan datos.

11. Crear las tablas y cargar los datos iniciales

Conectado como **POS en XEPDB1**, y únicamente si `user_tables` está vacío:

SQL*PLUS · SQL>

```
@"sql/01_esquema.sql"
@"sql/02_datos_iniciales.sql"
@"sql/03_verificar.sql"
```

Ejecuta los archivos de uno en uno. Si aparece un error, detente antes del siguiente: los scripts pueden cerrar SQL*Plus al detectar un fallo. El DDL de Oracle confirma cambios automáticamente y una creación parcial se conserva.

La verificación de una instalación nueva debe mostrar:

Dato	Esperado
Cuenta y contenedor	POS / XEPDB1
Tablas	15
Permisos / roles	9 / 4
Cajas / productos iniciales	3 / 5
Usuario de entrenamiento	admin activo
Índices de turno abierto	UX_TURNO_CAJA_ABIERTO y UX_TURNO_CAJERO_ABIERTO

Termina con:

SQL*PLUS · SQL>

```
EXIT
```

Si POS ya tiene tablas: no ejecutes SQL 01. Consulta `SELECT table_name FROM user_tables ORDER BY table_name;` y verifica el esquema existente. El script 02 agrega filas faltantes, pero no restablece precios, stock ni contraseñas; también restablece asociaciones iniciales faltantes. No lo uses para reiniciar prácticas.

Un clon nuevo con el esquema actual ya incorpora la apertura cero autorizada. Una base antigua se actualiza con `actualizar-bd.ps1` después de descargar el driver. No se actualiza borrando tablas ni ejecutando de nuevo SQL 00/01.

12. Descargar el driver y probar la conexión

En PowerShell, desde la raíz:

POWERSHELL · PS>

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\descargar-driver.ps1
Test-Path .\lib\ojdbc11.jar
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\conexion.ps1
```

El primer script descarga `ojdbc11.jar` 21.11 desde la publicación de Oracle en Maven Central y verifica su SHA-256. No instala Maven. `Test-Path` debe indicar `True`. La única biblioteca externa de la aplicación es ese driver.

La comprobación de conexión compila el código y solicita la clave de **POS**. En una base recién preparada debe mostrar:

RESULTADO O REFERENCIA

```
Compilacion correcta.
Usuario: POS; base: XEPDB1
Productos activos: 5
Conexion JDBC y consulta de productos: OK
```

Si agregaste o desactivaste productos, la cantidad puede ser distinta de cinco.

`config.properties` contiene valores no secretos:

ARCHIVO DE CONFIGURACIÓN

```
db.url=jdbc:oracle:thin:@//localhost:1521/XEPDB1
db.usuario=pos
efectivo.minimo=500.00
```

Si tu instalación usa otro puerto, crea `config.local.properties` junto al archivo base y escribe solo los valores no secretos que debas cambiar, por ejemplo `db.url=...`. Está ignorado por Git. La aplicación no lee contraseñas desde ese archivo y solicita la clave técnica en cada ejecución interactiva.

13. Compilar, abrir e ingresar como administrador

En PowerShell, dentro del clon:

POWERSHELL · PS>

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\compilar.ps1
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\iniciar.ps1
```

El primer comando debe indicar `Compilacion correcta.`. El segundo vuelve a compilar y abre la aplicación: en el uso diario basta con `iniciar.ps1`. Los `.ps1` se ejecutan con Windows PowerShell; no son programas de Java. `ExecutionPolicy Bypass` se aplica a ese proceso, no cambia la política global.

1. Cuando la consola solicite `Clave de Oracle para pos`, introduce la clave técnica de **POS de este equipo**.

2. En la ventana de login, utiliza la cuenta pública del ejercicio:

RESULTADO O REFERENCIA

```
Usuario: admin
Contraseña: Admin123!
```

3. Debe abrirse el menú con caja, usuarios, roles, asignaciones, productos y cajas. No crees otra cuenta admin: ya viene en los datos iniciales.

4. Conserva abierta la consola mientras uses la aplicación. Cuando cierres la aplicación y termine Java, PowerShell recuperará el prompt.

Si Main solicita la actualización de apertura cero, cierra la aplicación y ejecuta, usando la clave POS:

POWERSHELL · PS>

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\actualizar-bd.ps1
```

Este script conserva las tablas y los datos. No es necesario repetirlo cada vez que se abre la aplicación. No uses el botón Run del editor como primer método de inicio: la terminal permite introducir la clave técnica oculta.

14. Crear usuarios, conocer roles y añadir fotos

Con **admin**, abre **Administrar usuarios → Nuevo usuario** y crea las cuentas del escenario una por una. La contraseña la elige cada equipo y tiene al menos ocho caracteres; no se descarga desde GitHub.

Usuario	Nombre	Rol inicial
ana	Ana	CAJERO
luis	Luis	CAJERO
marta	Marta	SUPERVISOR

Los roles iniciales son CAJERO (nivel 1), SUPERVISOR (2), GERENTE (3) y ADMIN (4). Los permisos de roles y los extras se unen; el nivel es el mayor de sus roles. Un permiso extra no convierte al cajero en supervisor. Los cambios de permisos aplican al siguiente inicio de sesión.

Para cambiar de usuario usa **Cerrar sesión**. La autorización del supervisor durante un retiro o reversión no sustituye la sesión del cajero.

Para fotografías, entra como admin a **Catálogo de productos**, selecciona una fila y pulsa **Elegir fotografía...**. Admite JPG/JPEG o PNG hasta 10 MB, mantiene proporción y conserva el original. Los productos nuevos deben guardarse antes de asociar una foto.

Las copias se guardan en `datos/fotos/pos/<id>.png`. No vienen en el clone ni en `git pull`. Puedes elegir tus propias imágenes en este equipo. Para transportar fotos existentes necesitas la base correspondiente y sus IDs; copiar fotos a una base independiente no garantiza asociaciones correctas.

15. Demostrar administración, roles y catálogos

Usa una cuenta de práctica para conservar Ana, Luis y Marta con sus roles. Esta demostración enseña que un permiso extra no cambia el nivel de un usuario.

1. Como **admin**, abre **Administrar usuarios → Nuevo usuario**. Crea `prueba_caja` con rol CAJERO y una contraseña propia de al menos ocho caracteres.

2. En **Roles y permisos → Nuevo rol**, crea `APOYO_CAJA`, nivel **1**, y marca `VENTA_COBRAR`. Guarda y selecciona el rol para mostrar su matriz.

3. En **Asignar roles y permisos a usuarios**, selecciona `prueba_caja`. Conserva CAJERO, agrega APOYO_CAJA y marca `CATALOGO_ADMIN` como **Extra**. Los permisos heredados aparecen en gris y no se quitan desde los extras.

4. Guarda, cierra sesión e ingresa como `prueba_caja`. Productos y Cajas deben estar habilitados; Usuarios, Roles y Asignaciones deben estar deshabilitados.

5. En Productos crea `DEMO001`, nombre `Producto de práctica`, precio con impuesto **7.50**, impuesto **12.00**, stock **5** y Activo. Usa otro código si ya existe. Edita y desactiva: se conserva en el catálogo, sin ofrecerse en ventas.

6. En Cajas crea una con número libre, por ejemplo **4**, nombre `Caja de práctica` y activa. Abre su turno con **Q100**, sin hacer ventas.

7. Regresa al menú con la X del punto de venta e intenta desactivar esa caja: debe rechazarlo mientras tenga turno abierto. Cierra el turno declarando **Q100**; después podrás desactivar la caja.

8. Vuelve como admin y deja a `prueba_caja` solamente con CAJERO, sin APOYO_CAJA ni extras. Al siguiente login, Productos y Cajas vuelven a deshabilitarse. Si ya no la necesitas, desactiva esa cuenta desde Administrar usuarios.

No cambies los roles de tus propios administradores para demostrar restricciones. La aplicación impide modificar los roles propios y conceder niveles iguales o superiores. Una cuenta debe conservar al menos un rol. No borres registros con historial: usa desactivación.

Guarda las capturas de matriz, asignación, catálogos y menú con permisos. No es necesario ejecutar pruebas automáticas durante esta demostración.

16. Presentación: ventas, retiro, reversión y cierre

Empieza con Ana y Luis sin turnos abiertos. Usa Arroz a Q30 y Leche a Q20; necesitas al menos nueve arroces y cuatro leches. Verifica precios y stock antes de empezar. No agregues otras operaciones dentro de este turno.

1. Ana abre **Caja 1** con fondo **Q1000**.

2. Agrega **5 Arroces**, total Q150. Cobra efectivo, recibido **Q200**: cambio **Q50**. Anota el folio de esta primera venta.

3. Cobra **4 Leches**, total Q80, todo con tarjeta. Cambio Q0.

4. Cobra **4 Arroces**, total Q120, mixto: **Q50 tarjeta y Q70 efectivo**.

5. Usa la X del punto de venta para volver al menú y cierra sesión. El turno queda abierto; no pulses todavía **Cerrar turno**.

6. Luis comprueba que Caja 1 está ocupada y abre **Caja 2 con Q800**. Regresa al menú y cierra sesión sin cerrar su turno.

7. Ana entra de nuevo y pulsa **Abrir / continuar turno**. En **Corte intermedio** debe ver Q1220. Solicita retirar **Q500**, autorizado por Marta: quedan **Q720**.

8. En **Reversión**, busca el folio de la primera venta, selecciona Arroz, cantidad **1**, y escribe un motivo.

9. Solicita autorización. Luis debe ser rechazado aunque su clave sea correcta; Marta debe ser aceptada. El reembolso es **Q30 efectivo** y queda **Q690**.

10. En **Cerrar turno**, declara **Q680** y confirma. Guarda el reporte.

11. Si Luis no hizo ventas, cierra su turno declarando **Q800**.

Cálculo de Ana	Monto
Fondo inicial + cobros efectivos	Q1000 + Q220
Tarjeta, separada del efectivo	Q130
Retiro y reembolso	Q500 y Q30
Esperado / contado / diferencia	Q690 / Q680 / -Q10

El cierre debe quedar CERRADO. La devolución repone una unidad al stock y usa el precio histórico. Para repetir el escenario inicia turnos nuevos y verifica existencias. El lector de códigos no es obligatorio: usa el selector de productos y cantidad del punto de venta. Cobrar también se abre con F12.

17. Actualizar el código sin repetir la instalación

El propietario incorpora cambios a `main` mediante un PR. Tu compañero los recibe cuando ya estén en `main`; no necesita descargar ZIP ni clonar otra vez.

1. Cierra la aplicación Java. Entra al clon y comprueba el estado:

POWERSHELL · PS>

```
Set-Location "$env:USERPROFILE\Proyectos\SupermarketApp-Git"
git status
git branch --show-current
```

2. Si estás en **main** y el estado indica `working tree clean`:

POWERSHELL · PS>

```
git pull --ff-only origin main
```

3. Si indica `Already up to date`, ya tienes la versión publicada en main. Si aparecen cambios locales o estás en otra rama, conserva tus archivos y consulta antes de actualizar. No uses `reset --hard` ni `git clean` para forzar una actualización: pueden borrar trabajo local.

4. Revisa README por si el cambio requiere una migración SQL. Para la migración 04 existente usa `actualizar-bd.ps1` una vez si tu base es antigua, no SQL 00/01.

5. Ejecuta de nuevo `iniciar.ps1`. Ya compila el código actualizado.

`git pull` actualiza archivos de código. No actualiza Oracle, no cambia las claves locales, no importa ventas de otro compañero y no descarga las fotos ignoradas. La descarga inicial del driver tampoco se repite si ya está verificado.

Para revisar cambios sin incorporarlos todavía:

POWERSHELL · PS>

```
git fetch origin
git log -5 --oneline origin/main
```

18. Compartir una mejora mediante una rama y un PR

Este apartado es opcional para quien solo ejecutará el proyecto. Requiere permiso de escritura y configurar `user.name` y `user.email` del apartado 04.

1. Desde main limpio y actualizado, crea una rama con un nombre descriptivo:

POWERSHELL · PS>

```
git switch -c docs/mejora-explicacion
```

2. Como ejemplo, edita una explicación de `README.md` en VS Code y guarda.

3. Revisa exactamente lo que cambió y agrega solo ese archivo:

POWERSHELL · PS>

```
git status
git diff -- README.md
git add README.md
git commit -m "docs: aclarar instrucciones de ejecucion"
git push -u origin docs/mejora-explicacion
```

4. En GitHub abre **Compare & pull request**. Comprueba **base: main** y **compare: docs/mejora-explicacion**. Describe el cambio y crea el PR.

5. El propietario revisa los archivos y acepta **Merge pull request → Confirm merge**. Un commit local no basta: debe haberse enviado y luego integrado.

6. Después de integrar y con tus cambios guardados, vuelve a main:

POWERSHELL · PS>

```
git switch main
git pull --ff-only origin main
```

No agregues claves, dumps de Oracle, fotos privadas ni archivos generados. `.gitignore` ya excluye `out`, `bin-vscode`, JAR locales, configuración local y fotos. `git add README.md` es un ejemplo deliberadamente limitado; para otro cambio agrega los archivos concretos que revisaste. Si no tienes permisos, coordina una invitación o un fork con el propietario antes de intentar push.

19. Pruebas Java y verificaciones opcionales

La Fase 10 del documento exige un ejecutor propio en Java, sin JUnit ni Mockito, y al menos nueve reglas. Las clases en `test/` forman parte de la evidencia. Los `.ps1` de pruebas son atajos; no son un requisito de formato del documento. La demostración de ventanas puede hacerse manualmente como en el apartado 16.

Para ejecutar solamente las 37 pruebas de dominio, sin Oracle:

POWERSHELL · PS>

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\pruebas.ps1
```

Para las pruebas de base prepara **POS_TEST** una sola vez, independiente de POS. Desde PowerShell en la raíz:

POWERSHELL · PS>

```
sqlplus -L pos_test@//localhost:1521/XEPDB1
```

En SQL*Plus comprueba usuario, contenedor y cuenta vacía. Solo si hay cero tablas:

SQL*PLUS · SQL>

```
SHOW USER
SHOW CON_NAME
SELECT COUNT(*) AS tablas FROM user_tables;
```

SQL*PLUS · SQL>

```
@"sql/01_esquema.sql"
@"sql/02_datos_iniciales.sql"
@"sql/03_verificar.sql"
EXIT
```

Con POS_TEST listo, en PowerShell ejecuta `verificar-entrega.ps1` desde `scripts` igual que los demás scripts. Pide dos veces la clave POS_TEST y debe terminar con **37 de dominio + 35 de negocio + 28 de administración = 100 correctas**. Una base de pruebas antigua necesita su actualización 04 con `actualizar-bd.ps1 -Pruebas`. No uses POS para ejecutar estas suites de escritura.

20. Capturas y archivos para presentar

Prepara la carpeta en PowerShell:

POWERSHELL · PS>

```
New-Item -ItemType Directory -Path .\docs\evidencias\windows -Force
```

Usa Windows+Mayús+S. Abre la notificación de Recortes y guarda PNG; si solo queda en el portapapeles, pega en Paint y guarda. Captura las ventanas antes de confirmar o cerrarlas. Mantén las claves ocultas.

Archivo sugerido	Contenido
01-login.png	Login.
02-apertura.png	Cajero, caja y fondo.
03-carrito.png	Productos, cantidades y total.
04-pago.png	Pago mixto y cambio.
05-corte-intermedio.png	Retiro y resumen.
06-reversion.png	Venta, producto, cantidad y motivo.
07-autorizacion.png	Supervisor, clave oculta.
08-cierre.png	Esperado y contado.
09-usuarios.png / 09b-alta.png	Lista y formulario de usuario.
10-roles.png	Rol y matriz de permisos.
11-asignacion.png	Roles, heredados y extras.
12-productos.png / 12b-cajas.png	Los dos catálogos del apartado 12.
13-resultado.png	Q690 / Q680 / -Q10, CERRADO.
14-rechazo.png	Cajero rechazado como autorizador.

El material pide repositorio con commits por fases, README con ejecución y usuarios de prueba, diagrama ER, SQL, capturas de las doce pantallas y resultado integral. Los archivos `.md` son documentación: conserva README y el diagrama. Las otras guías ayudan, aunque no son necesarias para compilar.

Las imágenes en `docs/evidencias/final` son referencias de Linux/POS_TEST; no sustituyen capturas de la computadora donde tú ejecutaste la práctica.

21. Respaldo del proyecto, fotos y capturas

Cierra la aplicación y ejecuta desde la raíz:

POWERSHELL · PS>

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\respaldar.ps1
```

Se crea una carpeta con fecha dentro de `RespaldosPOS`, en tu carpeta de usuario. Incluye copia de archivos y ZIP verificado: código, scripts, SQL, pruebas, documentación y capturas guardadas en docs, driver y fotografías locales. Incluye configuración local e historial Git si están disponibles. Los cambios sin commit se conservan en los archivos copiados.

RESULTADO O REFERENCIA

```
RespaldosPOS/SupermarketApp-fecha-hora/
  proyecto/
  proyecto.zip
  SHA256-proyecto.txt
  LEEME-RESPALDO.txt
  base-datos/
```

La carpeta `base-datos` empieza vacía. **Este script no exporta Oracle**. Los archivos SQL crean una instalación, pero no respaldan ventas ni turnos. El apartado 22 explica cómo conservar tus datos reales.

Para otro destino externo al proyecto, por ejemplo una unidad disponible:

POWERSHELL · PS>

```
powershell -NoProfile -ExecutionPolicy Bypass `
  -File .\scripts\respaldar.ps1 -Destino "G:\RespaldosPOS"
```

El script no permite guardar el respaldo dentro del proyecto. Copia la carpeta completa a otra ubicación. Guárdala de forma privada porque puede contener configuración local y datos del ejercicio. Un ZIP de GitHub no sustituye este respaldo: no incluye driver, configuración ignorada, fotos ni datos Oracle.

22. Exportar los datos de Oracle con Data Pump

Mantén la aplicación cerrada. Necesitas la clave local de SYSTEM y la herramienta `expdp` incluida en la carpeta `bin` de Oracle. En PowerShell:

POWERSHELL · PS>

```
Get-Command expdp
$marcaPos = Get-Date -Format 'yyyyMMdd_HHmmss'
expdp system@localhost:1521/XEPDB1 schemas=POS `
  directory=DATA_PUMP_DIR "dumpfile=POS_$marcaPos.dmp" `
  "logfile=POS_$marcaPos.log"
```

Los acentos graves al final de las líneas permiten continuar el comando de PowerShell. No son comillas simples. Si prefieres, escribe `expdp ...` en una sola línea, separando sus parámetros con espacios.

Escribe la clave **SYSTEM** cuando se solicite. Espera el mensaje de exportación terminada correctamente y revisa el log. Conserva el error si falla; no borres el esquema ni consideres completo un respaldo con errores.

Para conocer dónde quedaron el dump y el log:

POWERSHELL · PS>

```
sqlplus -L system@localhost:1521/XEPDB1
```

En SQL*Plus:

SQL*PLUS · SQL>

```
SET LINESIZE 250
COLUMN directory_path FORMAT A180
SELECT directory_path FROM dba_directories
WHERE directory_name = 'DATA_PUMP_DIR';
EXIT
```

Abre la ruta mostrada en el Explorador. Copia, sin mover los originales, el `.dmp` y `.log` de la misma marca de tiempo a `base-datos` del respaldo. El dump de POS y `datos/fotos/pos` deben conservarse juntos por sus IDs de productos. POS_TEST no es necesario para conservar la práctica de la aplicación.

Para recuperar datos en otro equipo debe planificarse una importación con `impdp` y privilegios adecuados. No importes sobre la instalación funcional como ensayo antes de presentar. Una exportación correcta no demuestra por sí sola que ya se probó una restauración; esa validación se realiza aparte.

23. Explicar el código y abrir el diagrama ER

El proyecto usa **Java 21, Swing, JDBC puro y Oracle XE 21c**. No requiere Maven, Gradle ni frameworks. El driver JDBC es la única biblioteca externa de la aplicación. Los scripts PowerShell automatizan comandos de preparación.

Ruta	Responsabilidad y ejemplo
Main.java	Recibe la clave técnica, valida preparación y abre el login.
dominio/	Modelos: Usuario, Producto, Carrito, Turno y Venta.
ui/	Ventanas Swing: recogen datos y presentan resultados.
servicio/	Permisos, reglas y transacciones: VentaService, TurnoService.
dao/	SQL parametrizado con PreparedStatement y ResultSet.
util/	Configuracion, Conexion, Seguridad, Sesion y Dinero.
excepcion/	Errores de negocio y autorización.
test/	Ejecutores propios de pruebas.

Recorrido para explicar una venta: la ventana recoge el carrito; el servicio valida turno, pago y stock; los DAO guardan venta, detalles y pagos y actualizan existencias dentro de una transacción. Si todo funciona hay commit; si falla, rollback. Para dinero se usa BigDecimal con dos decimales y HALF_UP.

Las contraseñas de usuarios se guardan como hash PBKDF2 con sal aleatoria. No se guarda su texto original. El servicio vuelve a comprobar permisos, aunque la ventana también deshabilite botones.

El diagrama está en **docs/modelo-er.md**. Para verlo dibujado abre:

https://github.com/MJMontufar/SupermarketApp/blob/main/docs/modelo-er.md

GitHub representa Mermaid como un diagrama. En el editor puede verse su código fuente `erDiagram`. El modelo tiene 15 tablas: incluye usuarios/roles/permisos, productos, cajas/turnos, ventas/pagos/detalles, movimientos, reversiones y auditoría.

Descuentos, promociones y reporte gerencial por caja/cajero/hora son retos de ampliación del material y no están implementados. Sí existen resumen de turno, comprobantes y cierre con faltante/sobrante. La impresión física depende de la impresora local; no es un requisito para demostrar la vista previa.

24. Solucionar errores de Git, rutas y Java

Mensaje o síntoma	Qué revisar
`git` no se reconoce	Instala Git y abre una terminal nueva.
Repository not found / 404	Cuenta GitHub correcta, invitación aceptada y URL exacta.
Invalid username or token	Autentica con Credential Manager y navegador; no con contraseña HTTPS.
Destination path already exists	Entra al clon existente; revisa una carpeta parcial antes de renombrarla.
Not a git repository	Estás fuera de SupermarketApp-Git. Usa Set-Location.
Local changes would be overwritten	Respalda y conserva tus cambios; coordina su integración antes de pull.
El archivo `.ps1` no existe	Verifica la raíz y Test-Path; no ejecutes scripts/scripts desde scripts.
Java 17 o invalid target release 21	Revisa java, javac, JAVA_HOME y orden de PATH; se necesita JDK 21.
Paquetes com... no existen	Abre la raíz con src y test; compila todos los archivos con compilar.ps1.

Si la terminal está dentro de `scripts`, usa `./iniciar.ps1`; si está en la raíz, usa `./scripts/iniciar.ps1`. Esta guía usa siempre la raíz para evitar esa diferencia. En Windows puedes escribir las barras como en sus comandos.

Si solo ves README en el repositorio, confirma la rama y si el propietario ya incorporó el PR con el código. `git pull` no puede traer a main cambios que siguen únicamente en una rama no integrada.

Si los scripts usan un JDK que no existe, revisa `$env:JAVA_HOME`. Si esa variable está definida, los scripts la usan aunque `java` de PATH sea distinto. Corrige la variable y abre una terminal nueva. No desinstales herramientas ni edites los imports para resolver un problema de PATH.

25. Solucionar errores Oracle y comprobar la entrega

Error	Acción concreta
ORA-01017	Revisa cuenta y clave: SYSTEM, POS, POS_TEST y admin son distintas. Usa nueva conexión con `sqlplus -L`.
ORA-00942	Comprueba SHOW USER y SHOW CON_NAME. Puede faltar el esquema en POS o POS_TEST. No asumas que uno comparte tablas con el otro.
SP2-0310	SQL*Plus no encuentra el archivo. Sal, entra a la raíz del clon y vuelve a abrir SQL*Plus; para verificar usa `@"sql/03_verificar.sql"`.
SP2-0306	No pegues marcadores ni claves dentro de CONNECT. Sal y usa una conexión explícita desde PowerShell.
ORA-12541 / conexión rechazada	Revisa listener y puerto 1521. No modifiques usuarios para arreglar un listener detenido.
ORA-12514	Comprueba servicio XEPDB1 y que la base/PDB terminó de abrir. Conserva el mensaje para diagnóstico.
Faltan tablas o preparación parcial	Lista user_tables y conserva datos. No repitas SQL 01 sobre un esquema parcialmente creado.
Clave correcta pero login rechazado	Confirma usuario de la aplicación y estado activo; no introduzcas claves Oracle en el login.

Lista final antes de presentar:

- GitHub accesible y clon completo en una ruta corta.
- Java y javac 21; Oracle en ejecución; JDBC conectado a POS/XEPDB1.
- Admin entra y Ana, Luis y Marta tienen sus roles.
- Escenario ensayado; precios y stock listos para una nueva demostración.
- Capturas guardadas, README y diagrama ER disponibles.
- Respaldo de archivos y fotos; exportación POS correcta si se conservarán datos.

Arranque diario: abre PowerShell y ejecuta:

POWERSHELL · PS>

```
Set-Location "$env:USERPROFILE\Proyectos\SupermarketApp-Git"
powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\iniciar.ps1
```

Escribe la clave POS en consola y luego el usuario de la aplicación en la ventana. No necesitas reinstalar programas, clonar otra vez ni ejecutar SQL de creación cada día. Conserva la versión funcional que presentarás y sus respaldos.